WEJ LAB

Methods for Unsupervised Clustering

W. Evan Johnson, Ph.D.
Professor, Division of Infectious Disease
Director, Center for Data Science
Associate Director, Center for Biomedical Informatics and Health AI
Rutgers University – New Jersey Medical School

2026-10-28

RUTGERS HEALTH
Center for Data Science

Learning objectives

By the end of this lecture you should be able to:

- Choose a distance and decide whether to scale features
- Run and interpret hierarchical clustering and k-means
- Pick the number of clusters, and know the limits of the tools for it
- Check clusters against known labels
- Know when to reach for GMMs, DBSCAN, or graph-based (Leiden) clustering
- Make informative heatmaps

WEJ LAB
RUTGERS HEALTH
Center for Data Science

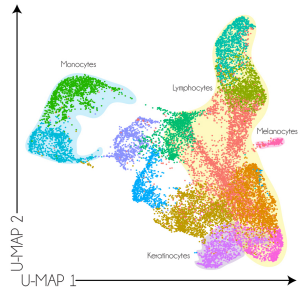
Supervised vs. Unsupervised Machine Learning

- **Supervised:** outcomes in the training set *supervise* the prediction algorithm
- **Unsupervised:** no outcomes; we look for groups in the data
- Unsupervised methods are often called **clustering** algorithms

WEJ LAB
RUTGERS HEALTH
Center for Data Science

Unsupervised Machine Learning

- Clustering is not always useful: from heights alone, we may not recover males and females
- But it is a powerful **exploratory** tool, e.g., cell types in single-cell data:

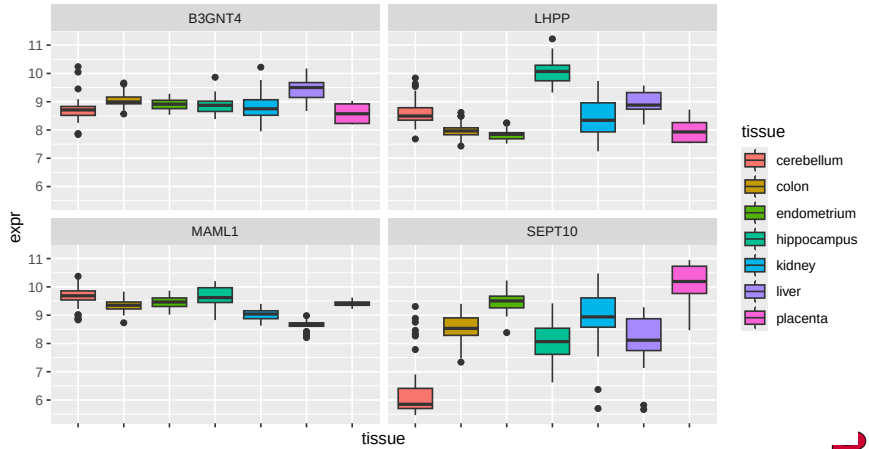


Look at your data first: boxplots

Gene expression for 189 samples from 7 tissues (500 genes).
Boxplots show each feature's center, spread, and outliers:

```
data("tissue_gene_expression")
dat <- data.frame(tissue = tissue_gene_expression$y,
                  tissue_gene_expression$x[, 1:4]) %>%
  pivot_longer(-tissue, names_to = "gene", values_to = "expr")
ggplot(dat, aes(tissue, expr, fill = tissue)) +
  geom_boxplot() + facet_wrap(~gene) +
  theme(axis.text.x = element_blank())
```

Look at your data first: boxplots



WEJ LAB
RUTGERS HEALTH
Center for Data Science

Choosing a distance

Every clustering method starts with a **distance**. The choice matters more than the algorithm:

- **Euclidean:** $\sqrt{\sum_j (x_{ij} - x_{kj})^2}$; sensitive to scale
- **Manhattan:** $\sum_j |x_{ij} - x_{kj}|$; less sensitive to outliers
- **Correlation:** $1 - r_{ik}$; similar *patterns* regardless of level. Common for gene expression

```
dist(x)                              # Euclidean
dist(x, method = "manhattan")
as.dist(1 - cor(t(x)))               # correlation
```

Should you scale?

- Features on large scales dominate Euclidean distance (e.g., age in years vs. a lab value in thousands)
- `scale(x)` gives each feature mean 0 and SD 1, so all count equally
- But scaling also amplifies **noisy, low-variance** features
- Gene expression: usually center, often scale, and filter low-variance genes first

```
d <- dist(scale(x))
```

WEJ LAB
RUTGERS HEALTH
Center for Data Science

Hierarchical clustering

1. Start with each observation as its own group
2. Merge the two closest groups
3. Recompute distances to the new group
4. Repeat until there is one group

WEJ LAB
RUTGERS HEALTH
Center for Data Science

Hierarchical clustering: movie ratings

Ratings of the top 50 movies from users with $\geq$ 25 ratings:

```
data("movielens")
top <- movielens %>% group_by(movieId) %>%
  summarize(n=n(), title = first(title)) %>%
  top_n(50, n) %>% pull(movieId)

x <- movielens %>% filter(movieId %in% top) %>%
  group_by(userId) %>% filter(n() >= 25) %>%
  ungroup() %>% select(title, userId, rating) %>%
  spread(userId, rating)

row_names <- str_remove(x$title, ": Episode") %>% str_trunc(20)
x <- x[,-1] %>% as.matrix()
x <- sweep(x, 2, colMeans(x, na.rm = TRUE))
x <- sweep(x, 1, rowMeans(x, na.rm = TRUE))
rownames(x) <- row_names
```

Missing data

31% of the movie $\times$ user matrix is missing (users didn't rate every movie).

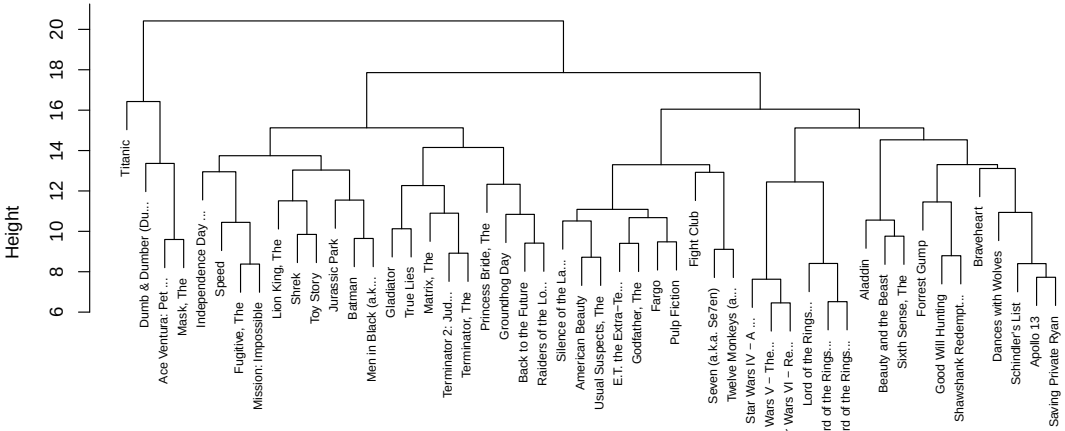
- `dist()` uses only the users who rated **both** movies, then rescales
- Fine for exploration, but distances rest on different numbers of raters
- Other options: impute, or filter to fewer movies/users with more overlap

Hierarchical clustering

Compute pairwise distances between movies, then cluster:

```
d <- dist(x)
h <- hclust(d)
```

Hierarchical clustering: dendrogram



WEJ LAB

RUTGERS HEALTH
Center for Data Science

Reading a dendrogram

- The height where two movies first join is their distance
- The three original *Star Wars* movies join below 8
- *Raiders of the Lost Ark* and *Silence of the Lambs* only join near 17

Question: how many groups would you cut this tree into? Why?

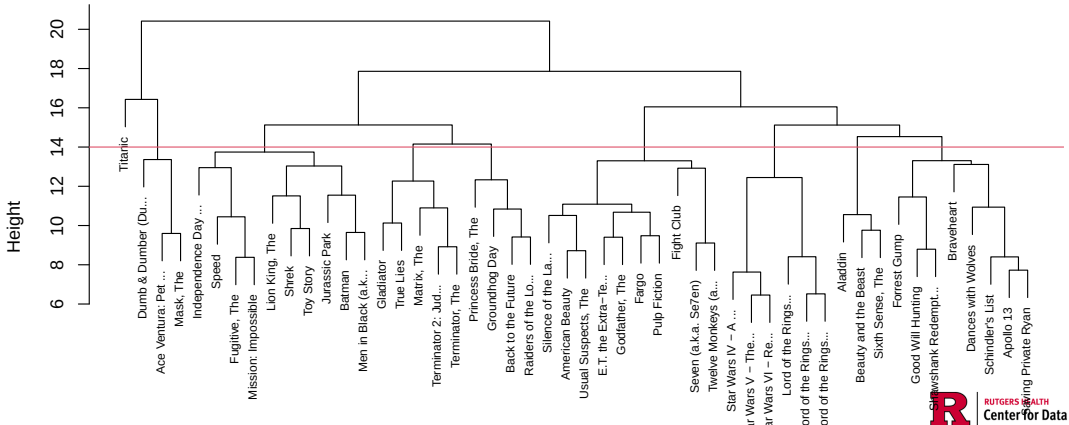
Linkage: distance between groups

- **Complete** (R default): max distance between members
- **Single:** min distance (tends to "chain")
- **Average:** mean distance
- **Ward's:** merge that least increases within-group variance

```
hclust(d, method = "ward.D2")
```

Cutting the tree

Either 1) set a maximum distance, or 2) set the number of groups:



WEJ LAB

RUTGERS HEALTH
Center for Data Science

Cutting the tree

cutree does either:

```
groups <- cutree(h, h = 14)  # maximum distance
table(groups)
```

```
## groups
##  1  2  3  4  5  6  7  8  9
##  3  3 10  8  4 10  5  6  1
```

```
groups <- cutree(h, k = 10)  # number of groups
table(groups)
```

```
## groups
##  1  2  3  4  5  6  7  8  9 10
##  3  3 10  8  4  6  4  5  6  1
```

Interpreting groups

Group numbers are arbitrary, so find groups by a member:

```
in_group <- function(g, title) names(g)[g == g[title]]
in_group(groups, "Star Wars IV - A ...")
```

```
## [1] "Lord of the Rings..." "Lord of the Rings..." "Lord of the Rings..."
## [4] "Star Wars IV - A ..." "Star Wars V - The..." "Star Wars VI - Re..."
```

```
in_group(groups, "Schindler's List")
```

```
## [1] "Apollo 13"            "Braveheart"           "Dances with Wolves"
## [4] "Forrest Gump"         "Good Will Hunting"    "Saving Private Ryan"
## [7] "Schindler's List"     "Shawshank Redempt..."
```

```
in_group(groups, "Ace Ventura: Pet ...")
```

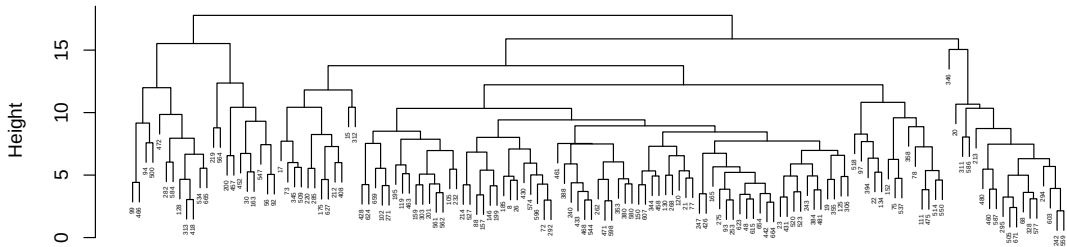
```
## [1] "Ace Ventura: Pet ..." "Dumb & Dumber (Du..." "Mask, The"
```

Sci-fi/fantasy franchises, prestige dramas, and Jim Carrey comedies.

Hierarchical clustering: raters

We can also cluster the raters (columns):

```
h_2 <- dist(t(x)) %>% hclust()
plot(h_2, cex = 0.35, main = "", xlab = "", sub = "")
```



WEJ LAB

RUTGERS HEALTH
Center for Data Science

K-means

Pre-specify k clusters, then iterate:

1. **Assign** each observation to the closest center
2. **Update** each center to the mean (**centroid**) of its cluster
3. Repeat until the centers stop changing

K-means: objective function

Minimize the **within-cluster sum of squares (WCSS)**:

$$\min_{S} \sum_{k=1}^{K} \sum_{\mathbf{x}_i \in S_k} ||\mathbf{x}_i - \boldsymbol{\mu}_k||^2$$

- S_k: observations in cluster k
- $\boldsymbol{\mu}_k$: centroid of cluster k

K-means: convergence

- Each step **decreases** the WCSS, so k-means always converges
- But only to a **local** minimum, which depends on the starting centers
- Fixes:
  - Many random starts (`nstart` in R) and keep the lowest WCSS
  - **k-means++** to choose better starting centers
- Assumes roughly round, similar-sized clusters

WEJ LAB
RUTGERS HEALTH
Center for Data Science

K-means in R

kmeans does not handle NAs. We fill them with 0, which (after centering) means **mean imputation**: an unrated movie gets the user's average.

```
x_0 <- x; x_0[is.na(x_0)] <- 0
set.seed(0)
k <- kmeans(x_0, centers = 10, nstart = 25)
groups <- k$cluster
table(groups)
```

```
## groups
##  1  2  3  4  5  6  7  8  9 10
##  3  8  3  1  3  7  9  3 10  3
```

K-means: groups

```
in_group(groups, "Star Wars IV - A ...")
```

```
## [1] "Star Wars IV - A ..." "Star Wars V - The..." "Star Wars VI - Re..."
```

```
in_group(groups, "Schindler's List")
```

```
## [1] "Apollo 13"            "Braveheart"           "Dances with Wolves"
## [4] "E.T. the Extra-Te..." "Gladiator"            "Godfather, The"
## [7] "Good Will Hunting"    "Saving Private Ryan"  "Schindler's List"
```

```
in_group(groups, "Pulp Fiction")
```

```
## [1] "American Beauty"      "Fargo"                "Fight Club"
## [4] "Pulp Fiction"         "Seven (a.k.a. Se7en)" "Silence of the La..."
## [7] "Twelve Monkeys (a..." "Usual Suspects, The"
```

Choosing k

No single right answer. Common tools:

- **Elbow plot:** WCSS vs. k; look for a bend
- **Silhouette width:** how much closer each point is to its own cluster than to the next one (-1 to 1; higher is better)
- **Gap statistic:** WCSS compared with data that have no clusters

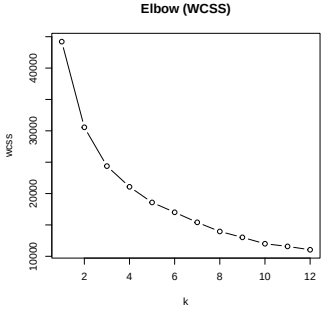
We'll try all three on the tissue data, where we know there are 7 tissues.

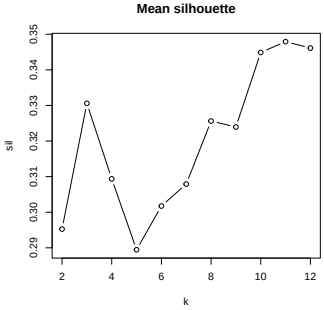
WEJ LAB
RUTGERS HEALTH
Center for Data Science

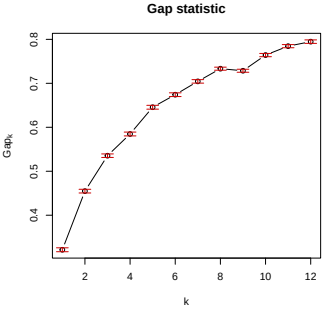
Choosing k: tissue data

```r
library(cluster)
xt <- sweep(tissue_gene_expression$x, 2, colMeans(tissue_gene_expression$x))
set.seed(1)
wcss <- sapply(1:12, function(k) kmeans(xt, k, nstart = 20)$tot.withinss)
sil  <- sapply(2:12, function(k)
  mean(silhouette(kmeans(xt, k, nstart = 20)$cluster, dist(xt))[, 3]))
gap  <- clusGap(xt, kmeans, K.max = 12, B = 20, nstart = 10)
par(mfrow = c(1, 3))
plot(1:12, wcss, type = "b", xlab = "k", main = "Elbow (WCSS)")
plot(2:12, sil,  type = "b", xlab = "k", main = "Mean silhouette")
plot(gap, main = "Gap statistic", xlab = "k")
```

Choosing k: tissue data







Choosing k: lessons

- The elbow is gradual; silhouette peaks at $k = 11$; the gap statistic picks $k = 8$. The truth is 7
- The criteria often **disagree**, and none knows the biology
- Real tissues contain sub-types, so "extra" clusters may be real
- Use these as guides, then check clusters against what you know

Checking clusters against known labels

Cross-tabulate clusters with the truth, and summarize with the **adjusted Rand index** (ARI: 1 = perfect agreement, 0 = chance):

```
set.seed(1)
km <- kmeans(xt, centers = 7, nstart = 25)$cluster
table(cluster = km, tissue = tissue_gene_expression$y)
```

```
##        tissue
## cluster cerebellum colon endometrium hippocampus kidney liver placenta
##       1         33     0           0           0      0     0         0
##       2          0     0           0           0      0     0         6
##       3          0    34           0           0      1     0         0
##       4          0     0          15           0      0     0         0
##       5          5     0           0          31      0     0         0
##       6          0     0           0           0      0    26         0
##       7          0     0           0           0     38     0         0
```

WEJ LAB
RUTGERS HEALTH
Center for Data Science

Checking clusters against known labels

```
mclust::adjustedRandIndex(km, tissue_gene_expression$y)
```

```
## [1] 0.9194083
```

- Colon, endometrium, kidney, liver, and placenta each get a clean cluster
- 5 cerebellum samples cluster with hippocampus (both brain)
- Agreement is high but not perfect, and k-means needed us to say $k = 7$
- Unsupervised methods find **structure**, which may not match your labels

Other Clustering Methods

Algorithm	Class	Key Features
Mixture model (GMM)	Model-based	Mixture of K distributions; **soft** assignments; fit with **EM**
DBSCAN	Density-based	Arbitrary shapes; flags **outliers**; no K needed
Louvain / Leiden	Graph-based	**Communities** in large networks by maximizing **modularity**
Affinity propagation	Message passing	Picks "exemplars" as centers; no K needed

Gaussian mixture models

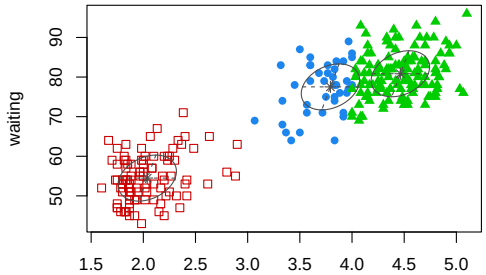
- Each cluster is a multivariate normal with its own mean and covariance
- Clusters can be **elliptical** and different sizes
- **Soft** assignment: each point gets a probability for each cluster
- `mclust` picks the number of clusters and covariance shape by **BIC**

Gaussian mixture models: Old Faithful

```
library(mclust)
fit <- Mclust(faithful)          # eruption length vs. waiting time
summary(fit)$G                   # clusters chosen by BIC
```

```
## [1] 3
```

```
plot(fit, what = "classification")
```



Gaussian mixture models: uncertainty

Soft assignments flag the points we are unsure about:

```
round(head(fit$z), 3)            # P(cluster | data) for each point
```

```
##     [,1]  [,2]  [,3]
## 1 0.972 0.000 0.028
## 2 0.000 1.000 0.000
## 3 0.997 0.000 0.003
## 4 0.000 1.000 0.000
## 5 0.012 0.000 0.988
## 6 0.003 0.997 0.000
```

```
sum(fit$uncertainty > 0.1)       # points with >10% uncertainty
```

```
## [1] 58
```

DBSCAN: density-based clustering

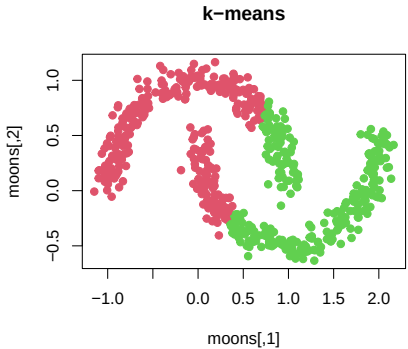
- A **core point** has at least `minPts` neighbors within radius `eps`
- Clusters grow by connecting core points; isolated points are **noise**
- Finds non-round shapes, and doesn't need k

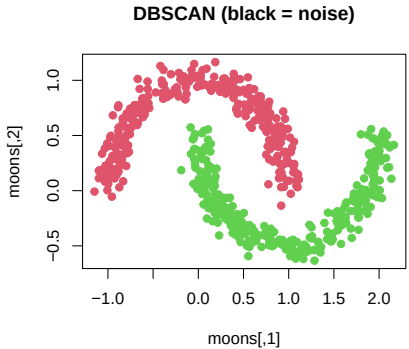
WEJ LAB
RUTGERS HEALTH
Center for Data Science

DBSCAN vs. k-means

```r
library(dbscan)
set.seed(2)
t <- runif(300, 0, pi)
moons <- rbind(cbind(cos(t), sin(t)),
               cbind(1 - cos(t), 0.5 - sin(t))) +
  matrix(rnorm(1200, sd = 0.08), ncol = 2)
km <- kmeans(moons, 2, nstart = 10)$cluster
db <- dbscan(moons, eps = 0.2, minPts = 5)$cluster   # 0 = noise
par(mfrow = c(1, 2))
plot(moons, col = km + 1, pch = 19, main = "k-means")
plot(moons, col = db + 1, pch = 19, main = "DBSCAN")
```

DBSCAN vs. k-means





Graph-based clustering: modularity

Standard for single-cell data (Seurat, Scanpy). Build a **nearest-neighbor graph**, then find communities that maximize **modularity**:

$$\mathcal{Q} = \frac{1}{2m} \sum_{i,j} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j)$$

- A_{ij}: edge weight between nodes i and j; m: total edge weight
- k_i: degree of node i; $\delta(c_i, c_j) = 1$ if same community

High $\mathcal{Q}$: dense connections within communities, sparse between.

Louvain and Leiden algorithms

Louvain (greedy; repeat until $\mathcal{Q}$ stops improving):

1. **Optimize:** move each node to the neighbor community with the largest gain in $\mathcal{Q}$
2. **Aggregate:** collapse each community into a single node; repeat

Leiden (Traag et al., 2019) adds a refinement step that guarantees connected communities. Now the default in Seurat and Scanpy.

Leiden clustering in R

```r
library(igraph); library(FNN)
pcs <- prcomp(xt)$x[, 1:20]                  # work in PC space
nn  <- get.knn(pcs, k = 10)$nn.index         # 10 nearest neighbors
g   <- graph_from_edgelist(cbind(rep(1:nrow(pcs), 10), as.vector(nn)),
                           directed = FALSE) %>% simplify()
set.seed(1)
cl  <- membership(cluster_leiden(g, objective_function = "modularity",
                                 n_iterations = -1))
```

WEJ LAB
RUTGERS HEALTH
Center for Data Science

Leiden clustering: results

```
table(cluster = cl, tissue = tissue_gene_expression$y)
```

```
##         tissue
## cluster cerebellum colon endometrium hippocampus kidney liver placenta
##       1         31     0           0           0      0     0        0
##       2          2     0           0           0     21     2        0
##       3          5     0           0          31      0     0        0
##       4          0    34           0           0      0     0        6
##       5          0     0          15           0      0     0        0
##       6          0     0           0           0     18     0        0
##       7          0     0           0           0      0    24        0
```

```
mclust::adjustedRandIndex(cl, tissue_gene_expression$y)
```

```
## [1] 0.7676888
```

Leiden clustering: lessons

- We never chose k: the **resolution** parameter controls how many communities
- Kidney splits in two, and placenta joins colon: more communities is not always better
- We'll use this again in the single-cell example next lecture

WEJ LAB
RUTGERS HEALTH
Center for Data Science

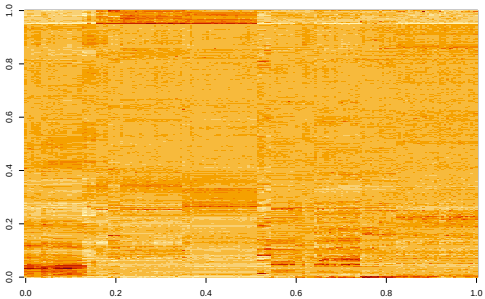
Heatmaps

- Plot the data matrix as colors
- Order rows and columns by hierarchical clustering

```
x <- xt                                    # centered tissue data
h_1 <- hclust(dist(x))
h_2 <- hclust(dist(t(x)))
```

Heatmaps

```
image(x[h_1$order, h_2$order])
```

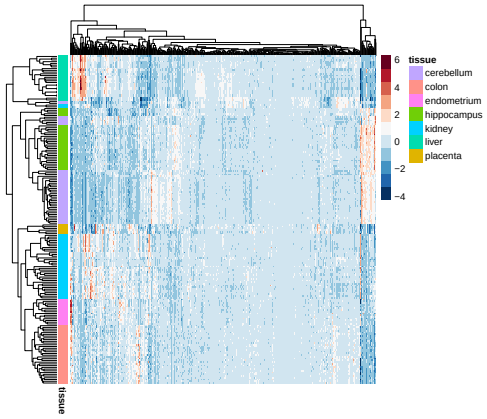


Heatmaps: pheatmap

pheatmap adds annotations and a legend. Note `scale`: base R's `heatmap()` rescales each **row** by default; be explicit.

```
library(pheatmap)
ann <- data.frame(tissue = tissue_gene_expression$y, row.names = rownames(x))
pheatmap(x, scale = "none", annotation_row = ann, show_rownames = FALSE,
         show_colnames = FALSE, color = rev(RColorBrewer::brewer.pal(11, "RdBu")))
```

Heatmaps: pheatmap

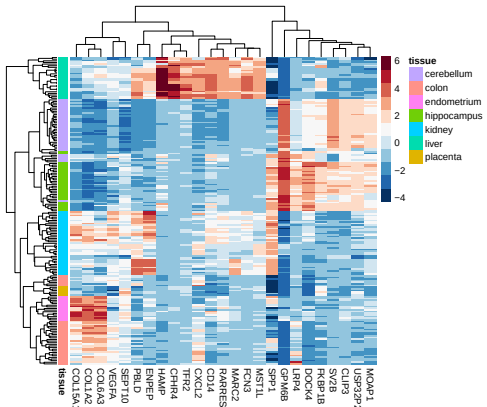


Filtering features

- Uninformative features add noise and hide clusters
- Simple fix: keep the **highest-variance** features

```
library(matrixStats)
o <- order(colSds(x), decreasing = TRUE)[1:25]
pheatmap(x[, o], scale = "none", annotation_row = ann,
         show_rownames = FALSE,
         color = rev(RColorBrewer::brewer.pal(11, "RdBu")))
```

Filtering features



Summary

- Distance and scaling choices drive the results; decide them on purpose
- Hierarchical: no k up front, gives a tree; k-means: fast, needs k, round clusters
- Choosing k: elbow, silhouette, gap are guides, not answers
- Validate: compare with known labels (table, ARI) and biology
- GMM (soft, elliptical), DBSCAN (shapes, noise), Leiden (graphs, single-cell)
- Heatmaps: annotate, and be explicit about scaling

Python companion

Python versions of these examples (`scikit-learn`, `scipy`, `seaborn`, `igraph`):

`python/week4/clustering.ipynb`

WEJ LAB
RUTGERS HEALTH
Center for Data Science

Session Info

```
sessionInfo()
```

```
## R version 4.5.2 (2025-10-31)
## Platform: aarch64-apple-darwin20
## Running under: macOS Tahoe 26.6.2
##
## Matrix products: default
## BLAS:   /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framework/Versions/A/libBLAS.dylib
## LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib;  LAPACK version 3.12.1
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## time zone: America/Denver
## tzcode source: internal
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
##  [1] matrixStats_1.5.0 pheatmap_1.0.13   FNN_1.1.4.1       igraph_2.3.4
##  [5] dbscan_1.2.5      mclust_6.1.2      cluster_2.1.8.2   dslabs_0.9.1
##  [9] lubridate_1.9.5   forcats_1.0.1     stringr_1.6.0     dplyr_1.2.1
## [13] purrr_1.2.2       readr_2.2.0       tidyr_1.3.2       tibble_3.3.1
## [17] ggplot2_4.0.3     tidyverse_2.0.0
##
## loaded via a namespace (and not attached):
##  [1] generics_0.1.4     stringi_1.8.9      hms_1.1.4          digest_0.6.39
```